

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
alekhya@Alekhyareddy: ~  
alekhya@Alekhyareddy:~$ nano main_loop.c  
alekhya@Alekhyareddy:~$ gcc main_loop.c -o main_loop  
alekhya@Alekhyareddy:~$ ./main_loop  
myshell> hello  
You entered: hello  
myshell> linux  
You entered: linux  
myshell> exit  
Exiting shell...  
alekhya@Alekhyareddy:~$ |
```

```
GNU nano 7.2  
#include <stdio.h>  
#include <string.h>  
  
int main()  
{  
    char command[100];  
  
    while (1)  
    {  
        printf("myshell> ");  
        fgets(command, sizeof(command), stdin);  
        command[strcspn(command, "\n")] = '\0';  
  
        if (strcmp(command, "exit") == 0)  
        {  
            printf("Exiting shell...\n");  
            break;  
        }  
  
        printf("You entered: %s\n", command);  
    }  
  
    return 0;  
}
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. Open()
2. Read()
3. Write()
4. Close()

open()

syntax: open(" filename.txt", modes, permissions)

Example: open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

#include <stdio.h>

#include <unistd.h>

int main()

{

 char buffer[100];

 int n;

 printf("Enter some text: ");

 n = read(0, buffer, sizeof(buffer) - 1);

 buffer[n] = '\0'; // Null-terminate the string

 printf("You entered: %s", buffer);

 return 0;

}

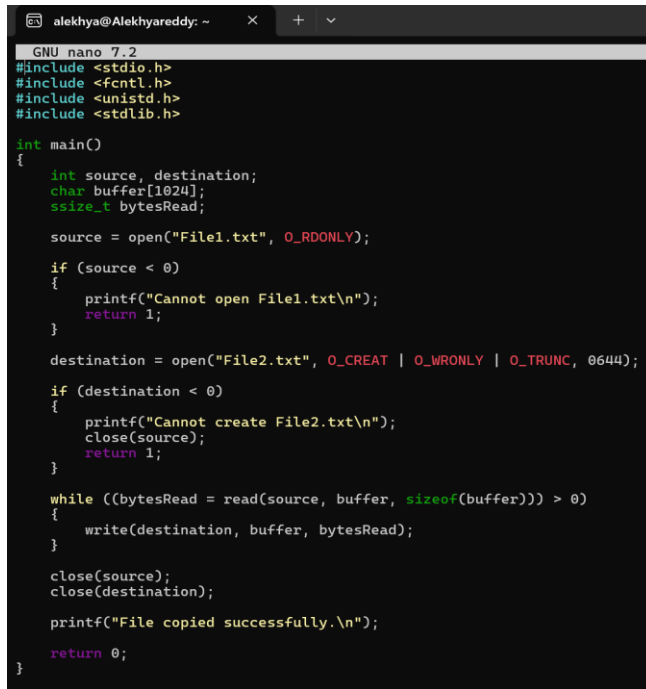
write()

Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

```
#include <unistd.h>

int main()
{
    write(1, "Hello World\n", 12);
    return 0;
}
```



```
GNU nano 7.2
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead;

    source = open("File1.txt", O_RDONLY);
    if (source < 0)
    {
        printf("Cannot open File1.txt\n");
        return 1;
    }

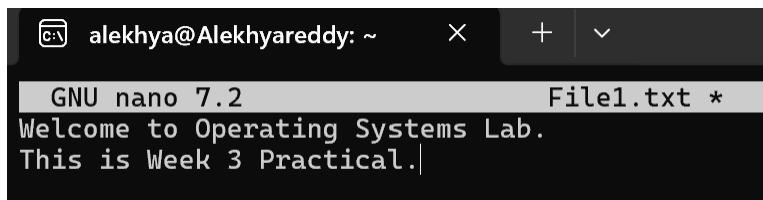
    destination = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (destination < 0)
    {
        printf("Cannot create File2.txt\n");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytesRead);
    }

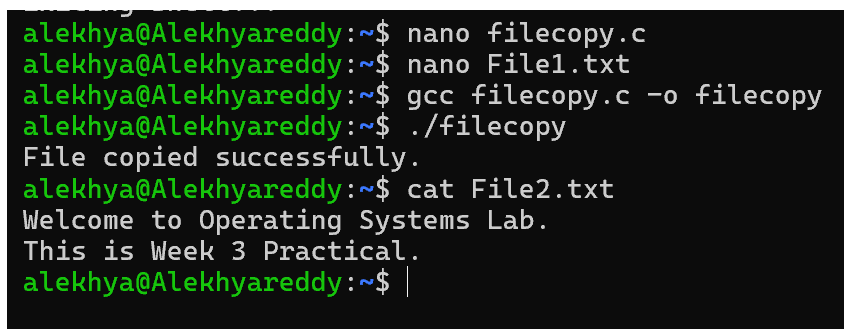
    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}
```



```
GNU nano 7.2 File1.txt *
Welcome to Operating Systems Lab.
This is Week 3 Practical.
```



```
alekhya@Alekhyareddy:~$ nano filecopy.c
alekhya@Alekhyareddy:~$ nano File1.txt
alekhya@Alekhyareddy:~$ gcc filecopy.c -o filecopy
alekhya@Alekhyareddy:~$ ./filecopy
File copied successfully.
alekhya@Alekhyareddy:~$ cat File2.txt
Welcome to Operating Systems Lab.
This is Week 3 Practical.
alekhya@Alekhyareddy:~$
```